```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Titanic.csv')   # exact filename adjust if needed

df.head()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Titanic.csv to Titanic (1).csv
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

```
df.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

Q1. Create a countplot to show the number of passengers by gender.

```
gender_count = df['Sex'].value_counts()
print(gender_count)
```

```
Sex
male      577
female    314
Name: count, dtype: int64
```

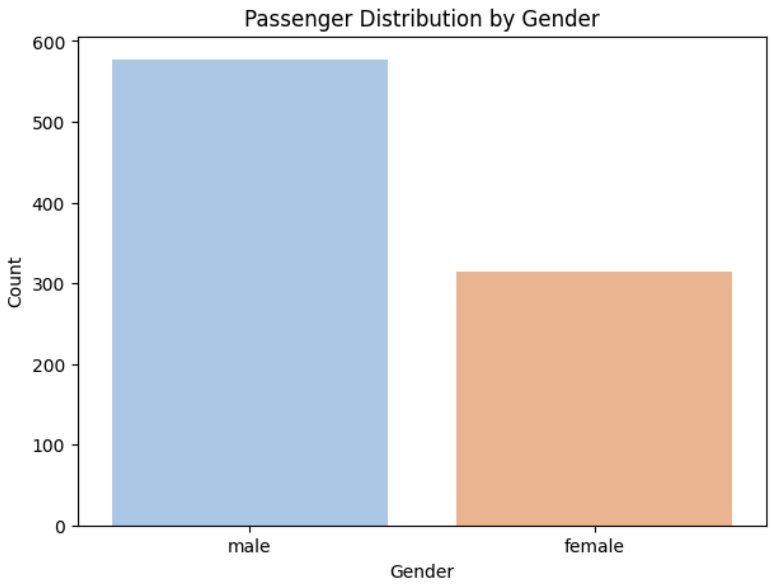
Visualization

```
plt.figure(figsize=(7,5))
sns.countplot(x='Sex', data=df, palette='pastel')
plt.title("Passenger Distribution by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
```

```
/tmp/ipykernel_2552/2677693169.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo

  sns.countplot(x='Sex', data=df, palette='pastel')
```



The output and chart show the number of male and female passengers aboard the Titanic. This helps understand the gender composition of passengers. If male count is higher, it indicates more male passengers traveled than female passengers.

Q2. Create a bar chart to show the number of passengers in each passenger class.

```
class_count = df['Pclass'].value_counts().sort_index()
print(class_count)
```

```
Pclass
1    216
2    184
3    491
Name: count, dtype: int64
```

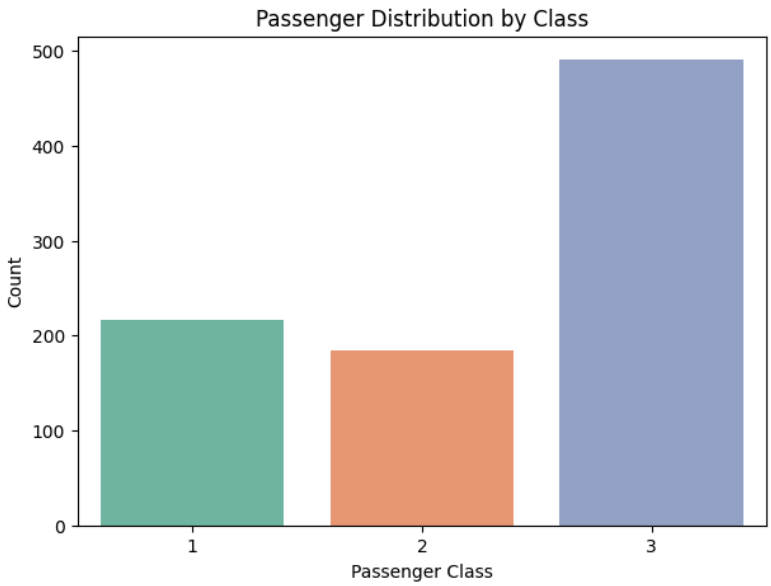
Visulisation

```
plt.figure(figsize=(7,5))
sns.countplot(x='Pclass', data=df, palette='Set2')
plt.title("Passenger Distribution by Class")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```

```
/tmp/ipykernel_2552/983637575.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo

  sns.countplot(x='Pclass', data=df, palette='Set2')
```



This chart shows how passengers were distributed among 1st, 2nd, and 3rd class. A higher count in one class indicates that most passengers belonged to that travel category.

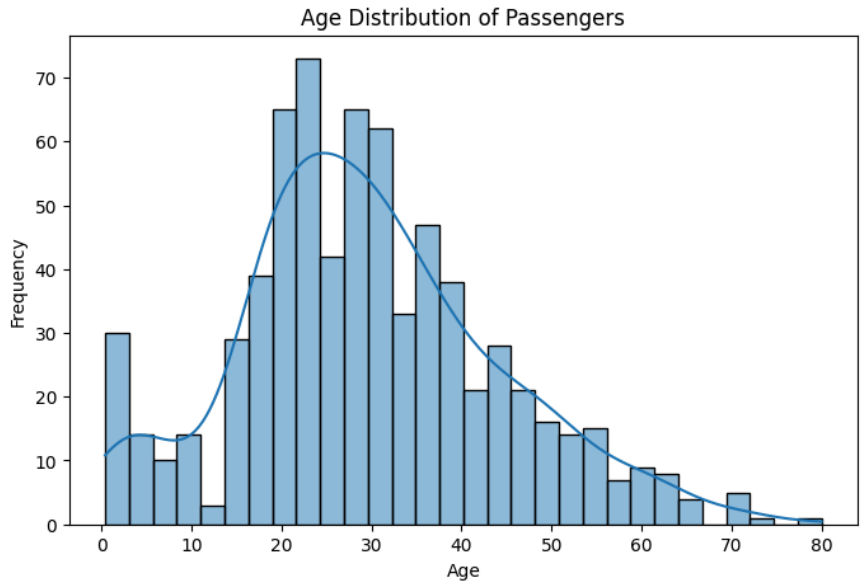
Q3. Create a histogram to show the distribution of passenger age.

```
print(df['Age'].describe())
```

```
count    714.000000
mean      29.699118
std       14.526497
min        0.420000
25%       20.125000
50%       28.000000
75%       38.000000
max       80.000000
Name: Age, dtype: float64
```

Visualization

```
plt.figure(figsize=(8,5))
sns.histplot(df['Age'].dropna(), bins=30, kde=True)
plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



This histogram displays the age distribution of Titanic passengers. It helps identify the dominant age group onboard and whether passengers were mostly young, middle-aged, or elderly.

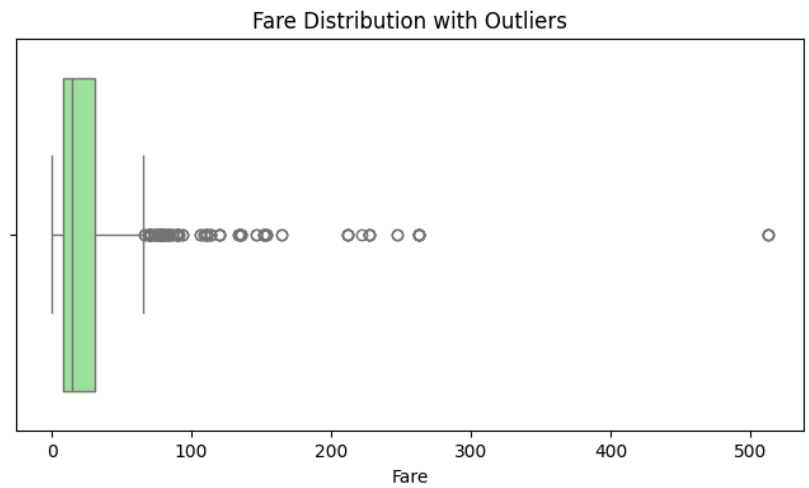
Q4. Create a boxplot to detect outliers in the fare column.

```
print(df['Fare'].describe())
```

```
count    891.000000
mean      32.204208
std       49.693429
min        0.000000
25%        7.910400
50%       14.454200
75%       31.000000
max      512.329200
Name: Fare, dtype: float64
```

Visualization

```
plt.figure(figsize=(8,4))
sns.boxplot(x=df['Fare'], color='lightgreen')
plt.title("Fare Distribution with Outliers")
plt.xlabel("Fare")
plt.show()
```



The boxplot highlights outliers in fare values. Extreme fare values indicate premium ticket purchases and luxury accommodations.

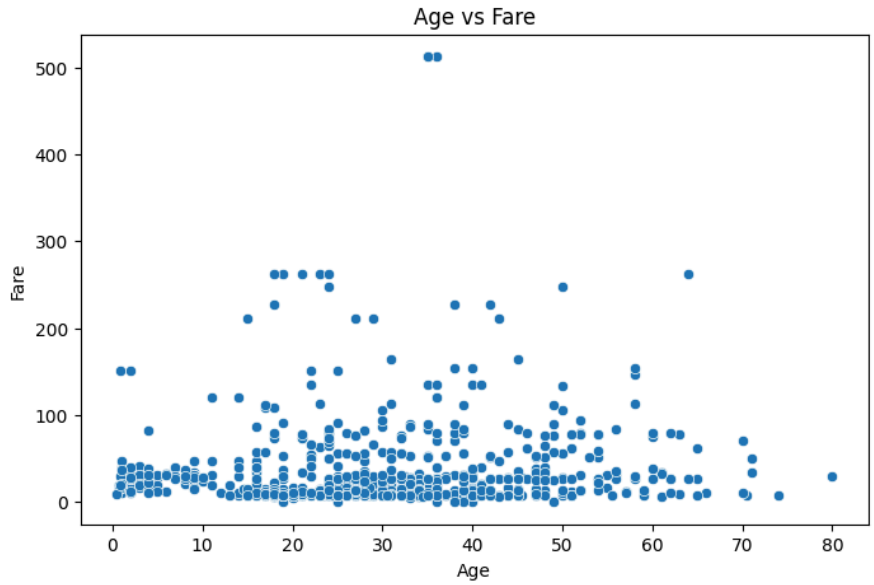
Q5a. Show relationship between Age and Fare.

```
print(df[['Age','Fare']].corr())
```

```
           Age      Fare
Age   1.000000  0.096067
Fare  0.096067  1.000000
```

Visualization

```
plt.figure(figsize=(8,5))
sns.scatterplot(x='Age', y='Fare', data=df)
plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.show()
```



This scatterplot helps analyze whether passenger age influenced ticket fare. A weak/no visible trend suggests age may not strongly affect fare paid.

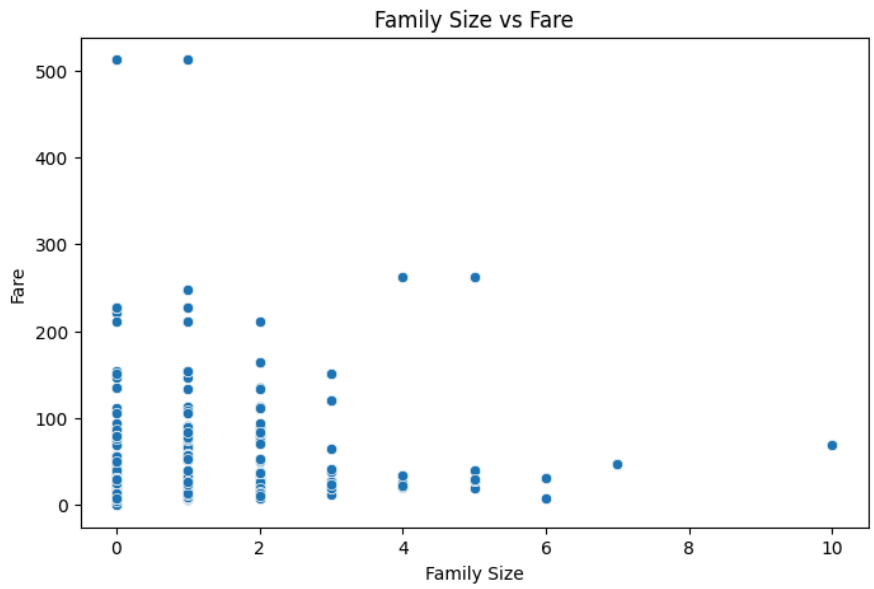
Q5b. Analyze the relationship between family size and fare.

```
df['FamilySize'] = df['SibSp'] + df['Parch']
print(df[['FamilySize','Fare']].corr())
```

```
            FamilySize      Fare
FamilySize    1.000000  0.217138
Fare          0.217138  1.000000
```

Visualization

```
plt.figure(figsize=(8,5))
sns.scatterplot(x='FamilySize', y='Fare', data=df)
plt.title("Family Size vs Fare")
plt.xlabel("Family Size")
plt.ylabel("Fare")
plt.show()
```



This visualization shows whether larger families paid higher fares. A positive trend may indicate higher total fare for bigger families/groups.

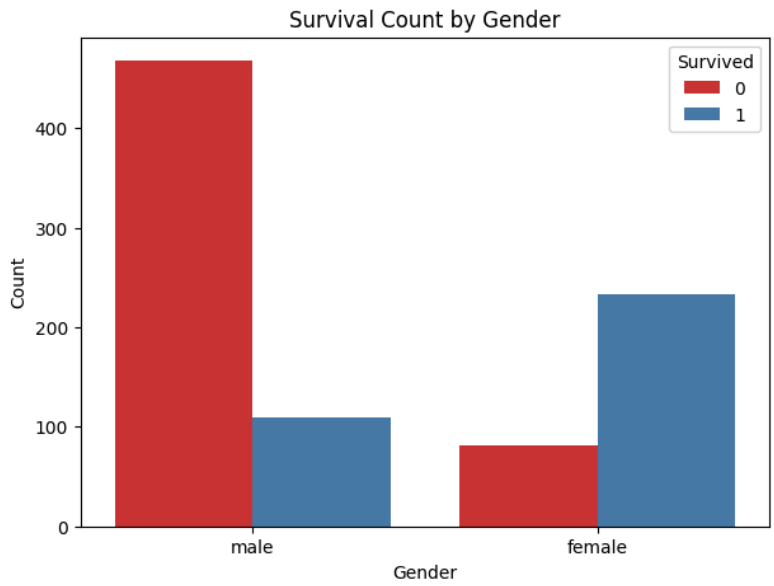
Q6. Create a countplot to show survival count by gender.

```
survival_gender = pd.crosstab(df['Sex'], df['Survived'])
print(survival_gender)
```

```
Survived    0    1
Sex
female     81  233
male      468  109
```

Visualization

```
plt.figure(figsize=(7,5))
sns.countplot(x='Sex', hue='Survived', data=df, palette='Set1')
plt.title("Survival Count by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
```



This chart compares survival outcomes for male and female passengers. It helps determine whether gender had an impact on survival probability.

Q7. Create a bar chart to show survival by passenger class.

```
survival_class = df.groupby('Pclass')['Survived'].mean()
print(survival_class)
```

```
Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

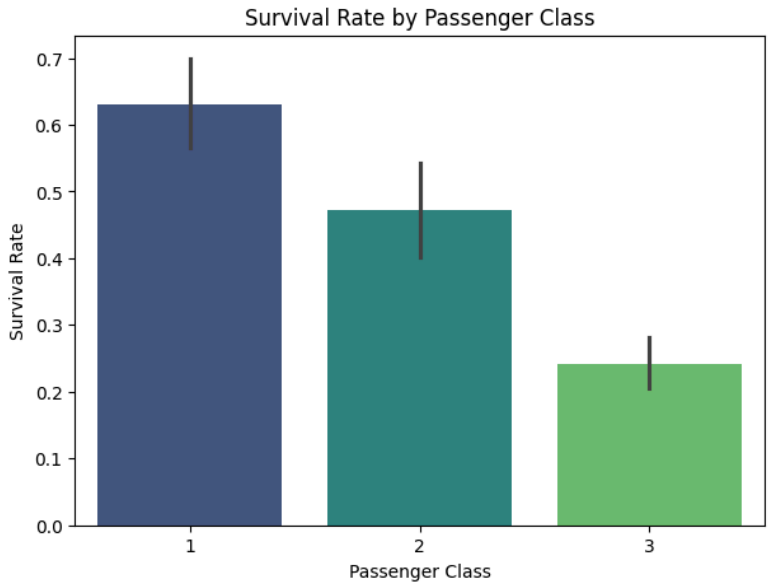
Visualization

```
plt.figure(figsize=(7,5))
sns.barplot(x='Pclass', y='Survived', data=df, palette='viridis')
plt.title("Survival Rate by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Survival Rate")
plt.show()
```

```
/tmp/ipykernel_2552/1994290869.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo

  sns.barplot(x='Pclass', y='Survived', data=df, palette='viridis')
```



This chart shows how survival probability varied by passenger class. Higher-class passengers may have had better access to lifeboats and evacuation routes.

Q8. Create a heatmap to show correlation between numerical variables.

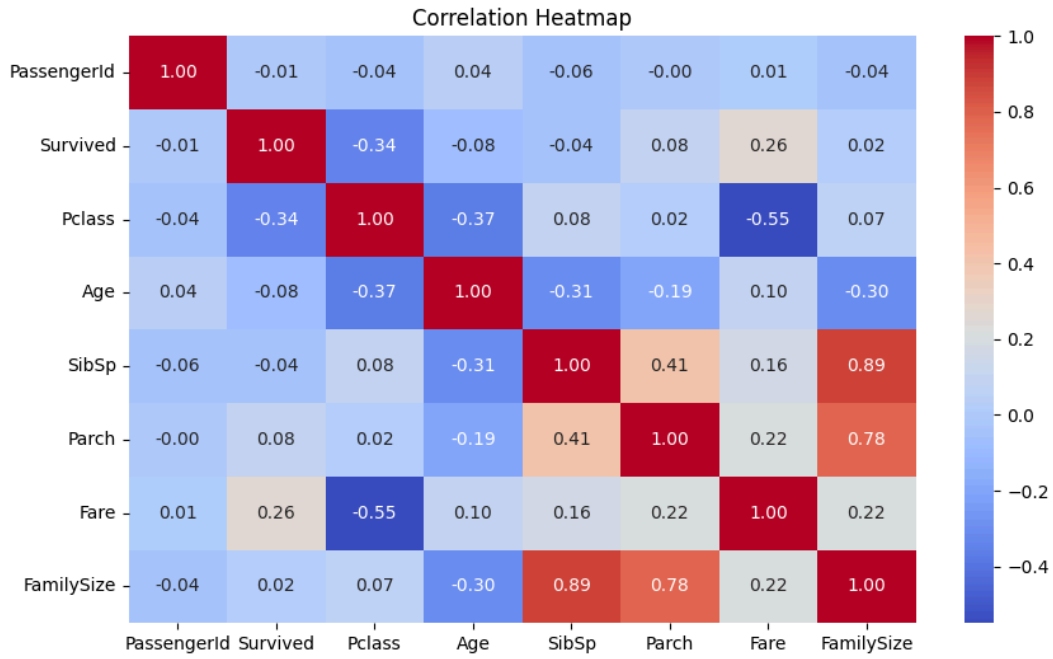
```
print(df.corr(numeric_only=True))
```

```
             PassengerId  Survived    Pclass       Age     SibSp     Parch  \
PassengerId     1.000000 -0.005007 -0.035144  0.036847 -0.057527 -0.001652
Survived       -0.005007  1.000000 -0.338481 -0.077221 -0.035322  0.081629
Pclass         -0.035144 -0.338481  1.000000 -0.369226  0.083081  0.018443
Age             0.036847 -0.077221 -0.369226  1.000000 -0.308247 -0.189119
SibSp          -0.057527 -0.035322  0.083081 -0.308247  1.000000  0.414838
Parch          -0.001652  0.081629  0.018443 -0.189119  0.414838  1.000000
Fare            0.012658  0.257307 -0.549500  0.096067  0.159651  0.216225
FamilySize     -0.040143  0.016639  0.065997 -0.301914  0.890712  0.783111

                 Fare  FamilySize
PassengerId  0.012658   -0.040143
Survived     0.257307    0.016639
Pclass      -0.549500    0.065997
Age          0.096067   -0.301914
SibSp        0.159651    0.890712
Parch        0.216225    0.783111
Fare         1.000000    0.217138
FamilySize   0.217138    1.000000
```

Visualization

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap")
plt.show()
```



The heatmap shows relationships between numerical variables. Values close to 1 or -1 indicate stronger positive or negative relationships respectively.

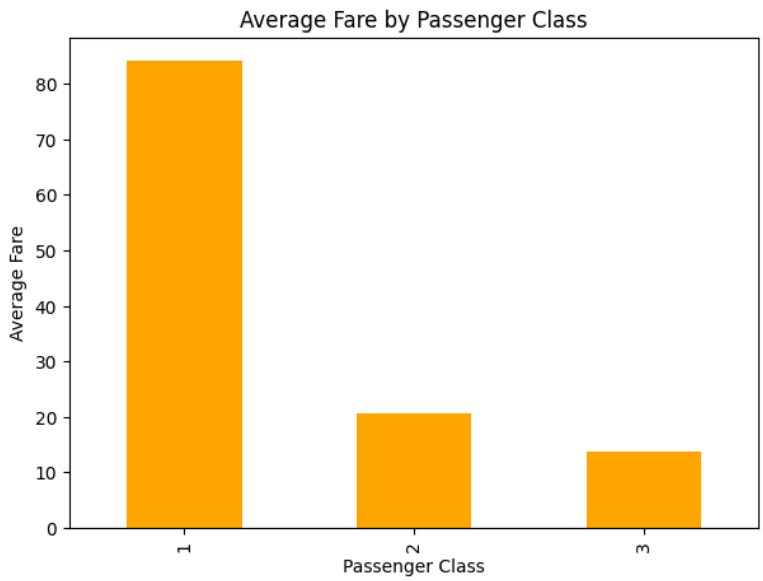
Q9. Identify which passenger class paid the highest fare on average.

```
fare_by_class = df.groupby('Pclass')['Fare'].mean()
print(fare_by_class)
print("Highest Average Fare Paid By Class:", fare_by_class.idxmax())
```

```
Pclass
1    84.154687
2    20.662183
3    13.675550
Name: Fare, dtype: float64
Highest Average Fare Paid By Class: 1
```

Visualization

```
plt.figure(figsize=(7,5))
fare_by_class.plot(kind='bar', color='orange')
plt.title("Average Fare by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Average Fare")
plt.show()
```



This chart shows average fare paid by passengers in each class. The class with the highest value indicates premium ticket pricing.

```
print("Q10. Final Insights\n")
```

```
print("1. Male passengers were more in number than female passengers, indicating higher male representation aboard the Titanic.")
print("2. Majority of passengers belonged to 3rd class, showing that lower-class travel was most common.")
print("3. Most passengers were young adults based on the age distribution.")
print("4. Fare distribution contains several high-value outliers, indicating premium/luxury ticket purchases.")
print("5. Female passengers had significantly higher survival rates compared to male passengers.")
print("6. First-class passengers had better survival probability than lower-class passengers.")
```